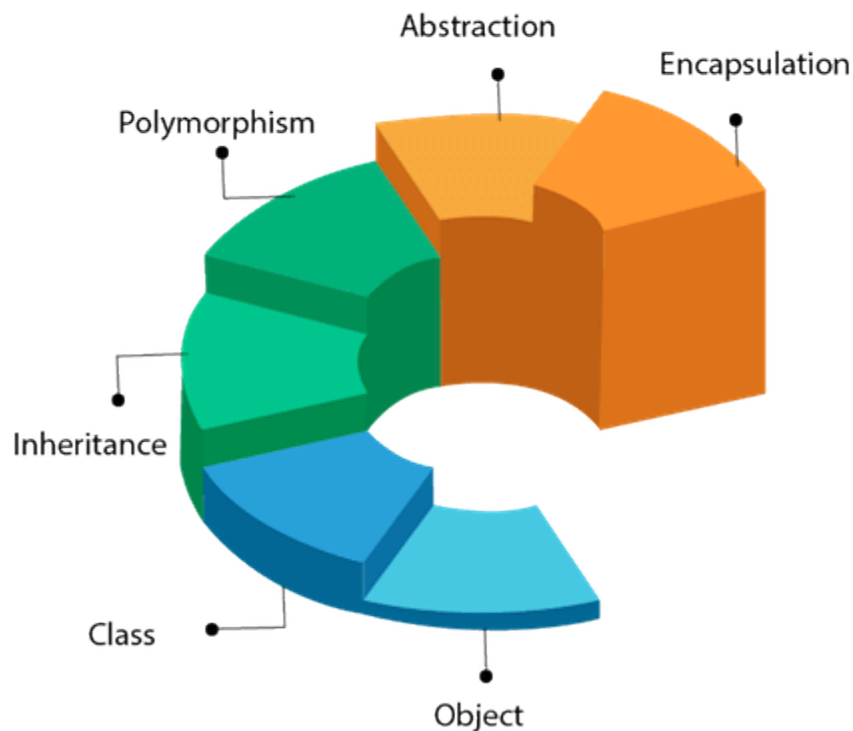


Object-Oriented Programming (OOP) is a programming model that structures software around "objects," which bundle data (properties) and functions (methods) that operate on that data, mirroring real-world entities to create organized, reusable, and maintainable code, with key principles like [Encapsulation](#), [Inheritance](#), and [Polymorphism](#). It focuses on objects interacting with each other, making complex systems easier to manage and develop collaboratively, especially for large applications.

Core Concepts

- **[Class](#)**: A blueprint or template that defines the properties and methods for a type of object (e.g., a Car class).
- **[Object](#)**: An instance of a class, containing its own data (e.g., a specific Volvo car).
- **[Encapsulation](#)**: Bundling data and methods within an object, hiding internal details and controlling access.
- **[Inheritance](#)**: Allows a new class (subclass) to inherit properties and methods from an existing class (superclass).
- **[Polymorphism](#)**: The ability for objects to take on many forms, allowing one interface to represent different underlying data types or behaviors.

OOPs (Object-Oriented Programming System)



The 4 Main Blocks Of Object-Oriented Programming

Classes – This is a blueprint for creating objects; it provides initial value for states and implementations of behavior. The class defines the nature of a future object. You can interpret class as a concept while the object is the reality or the embodiment of that concept.

Objects – An object is a data field with unique attributes and behavior. These are instances of a class created with certain data. Objects have the flexibility to correspond to real-world objects or abstract entities.

Methods – This defines the behavior of the objects created from the class. To put it simply, the method is an action that an object is able to perform. These are functions defined in a class that describes what the object will do.

Attributes – These are the characteristics of the class that aids in separating it from other classes. It is what makes them unique and identifiable.

Key Principles (The Four Pillars)

1. **Abstraction:** Hiding complex implementation details and showing only essential features.
2. **Encapsulation:** Protecting data by restricting direct access, as described above.
3. **Inheritance:** Promoting code reuse by deriving new classes from existing ones.
4. **Polymorphism:** Enabling objects to perform different actions based on their type, often through method overriding or interfaces.

Benefits

1. **Reusability:** Inheritance and classes promote using code multiple times.
2. **Maintainability:** Clear structure makes code easier to debug, update, and manage.
3. **Scalability:** Better suited for large, complex projects that grow over time.
4. **Modularity:** Breaks down complex systems into smaller, manageable objects.

Popular OOP Languages

Languages like Java, C++, Python, C#, Ruby, and JavaScript heavily support OOP, making it a dominant paradigm for modern software development.

To write and run your first Java program, you need the **Java Development Kit (JDK)** installed and a plain text editor (like Notepad), TextEdit, or an Integrated Development Environment (IDE) like Eclipse or IntelliJ IDEA.

Step 1: Write the Java Source Code

Open your text editor and type the following code exactly as it appears. This program will print "Hello, World!" to the console.

```
java

public class HelloWorld {

    public static void main(String[] args) {

        System.out.println("Hello, World!");

    }

}
```

- **public class HelloWorld:** Defines a class named HelloWorld, which must match the file name.

- **public static void main(String[] args):** This specific method is the entry point for all Java applications; the program starts executing from here.
- **System.out.println("Hello, World!");**: This statement prints the text "Hello, World!" to the standard output (your screen).

Step 2: Save the File

Save the file with the name **HelloWorld.java**.

- Crucially, the file name must match the class name defined in the code, including capitalization.
- When saving in a basic text editor like Notepad, make sure to select **"All Files"** in the "Save as type" dropdown menu to prevent it from being saved as a .txt file.

Step 3: Compile the Program

1. Open your computer's **command prompt** or terminal.
2. Navigate to the directory where you saved your HelloWorld.java file using the cd command (e.g., cd C:\Users\YourName\Documents if you saved it in Documents).
3. Compile the source code using the javac command followed by your file name:

```
javac HelloWorld.java
```

4. If there are no errors, this command will create a new file in the same directory named `HelloWorld.class`. This `.class` file contains the bytecode that the Java Virtual Machine (JVM) can execute.

Step 4: Run the Program

1. In the same command prompt window, run the compiled program using the `java` command followed by the class name (without the `.class` or `.java` extension):
2. The output `Hello, World!` will be displayed in the console.

Tips and Best Practices

- **Case Sensitivity:** Java is case-sensitive. Ensure that you use the correct capitalization for class names, method names, and file names.
- **File Naming:** The filename must match the public class name exactly, including capitalization.

- **Environment Setup:** Ensure that Java Development Kit (JDK) is installed and configured correctly on your system. You can verify the installation by running `java -version` and `javac -version` in your command prompt or terminal.
- **Use Comments:** Add comments to your code to improve readability and maintainability. For example:

```
// This is a single-line comment
```